

Animated Pohela Boishakh Celebration Scene using OpenGL (Grp-G)

Group Members and Contribution

Name	ID	Contribution
Fihab Ifty	23-51180-1	Sky, day-night mode, stars, rain, cloud, boat, road, ferries wheel, fair stall, overall integration
Kaniz Nawrin Ahmed	23-52904-2	Road, vehicles, sky, bird, village home
Umme Fahmida Sultana	23-52918-2	River, boats, ghat, sea, sun, trees
Eyamin Khan Emon	23-52922-2	Ferris wheel, stalls, people, kite, vehicle, balloons, boat, moon, overall interaction

Table 1: Group members and their Name and Contribution

1. Introduction

1.1 Scenario Explanation

Pohela Boishakh is the Bengali New Year and one of the most important cultural festivals of Bangladesh. On this day, people celebrate with joy by visiting fairs, enjoying traditional food, riding Ferris wheels, watching kite flying, boat rides, cultural programs, and social gatherings. The environment becomes lively with decorated roads, busy rivers, colorful stalls, and happy crowds.

This project presents a 2D animated Pohela Boishakh village and mela scene using OpenGL. The scene includes roads connected with side roads, rivers with moving boats, ghats, houses near hills, trees, clouds, sun, moon, stars, rain, Ferris wheels, food stalls, people, kites, and balloons. Each component is designed using basic OpenGL primitives and combined to form a complete interactive environment.

1.2 Motivation

The motivation behind this project is to learn computer graphics in a practical way. Instead of drawing isolated objects, we aimed to create a complete animated world where multiple elements interact with each other. We also wanted to understand how transformations such as translation, rotation, and scaling are applied, and we ensured these concepts are properly implemented in our project.

Another motivation was to represent Bangladeshi culture through technology. Most graphics projects use generic scenes, but this project focuses on a culturally meaningful scenario. Boats in the river, kite flying in the sky, and Ferris wheels in the mela are common sights during Pohela Boishakhi.

1.3 Novelty of the project

The novelty of this project lies in its interactivity and cultural storytelling. The user can switch between day and night, turn rain on and off, control Ferris wheel rotation, and observe different animations.

The project also includes dynamic elements like moving clouds, boats, kites, and falling rain. The platform used for this project is OpenGL with C++. OpenGL is a low-level graphics library

that allows direct control over rendering. GLUT is used for window creation, keyboard handling, and timers. This platform is widely used in computer graphics education and is suitable for learning rendering concepts.

1.4 Background of the Scenario

The scene is inspired by a village-side Pohela Boishakh mela. Typically, such areas include roads with moving vehicles, rivers with boats and ghats, houses located near hills, open fields with trees, and a festive mela environment containing stalls, ferris wheels, balloons, and people. These elements are commonly observed during Bengali cultural celebrations. In this project, each object has been carefully placed based on real-life village layouts and cultural observation to maintain realism and cultural relevance.

1.5 Background of the Platform Used

The project is developed using the C++ programming language along with the OpenGL graphics library. GLUT (or FreeGLUT) is used for window creation, input handling through keyboard and mouse, and timer-based animation control. OpenGL is widely used in computer graphics education because it offers low-level control over drawing primitives, transformations, and rendering processes. This makes it an effective platform for learning fundamental concepts such as coordinate systems, object transformation, animation, and interactive graphics.

2. Related Works

Previous works in computer graphics have extensively explored the use of two-dimensional rendering techniques to visualize real-life environments and cultural scenarios. Many studies focus on using OpenGL as a foundational graphics library because of its simplicity, flexibility, and suitability for educational and simulation-based projects. In particular, OpenGL-based festival and cultural scene simulations have gained popularity as they allow developers to combine static scene modeling with basic animations to create engaging visual storytelling experiences [1].

Several works demonstrate how OpenGL primitives such as points, lines, triangles, and quadrilaterals can be used to construct complex scenes from simple geometric components. In cultural festival scene simulations, static objects such as stalls, decorations, houses, and human figures are often drawn using combinations of these primitives. Coordinate systems and transformation techniques play a vital role in positioning these objects correctly within the scene. Studies focusing on OpenGL

coordinate systems explain how translation, scaling, and rotation operations enable developers to control object placement and perspective effectively [1]. These techniques form the backbone of many festival and village environment visualization projects.

Rural and village environment visualization has also been widely explored in computer graphics research. Many studies aim to recreate village landscapes by modeling houses, trees, roads, rivers, and open fields in a two-dimensional or semi-three-dimensional space. Such works emphasize scene composition, object layering, and depth illusion to create a sense of realism [2]. By carefully arranging foreground, middle-ground, and background elements, these projects simulate natural village settings despite being rendered in 2D. This approach closely aligns with the concept of storytelling through environment design, where visual elements convey cultural and geographical context.

Animation techniques form another important area of research in 2D graphics-

based projects. One common animation scenario involves simulating boats moving across a river surface. Prior works demonstrate how translation transformations combined with timer-based animation loops can create smooth and continuous motion of objects along predefined paths [3]. These studies highlight the importance of frame updates and controlled movement speed to maintain visual consistency. Boat animation projects are particularly useful in understanding how environmental interaction, such as movement on water, can be represented using simple mathematical calculations in OpenGL.

In addition to water-based animations, several studies focus on simulating aerial motion, such as kite flying. Kite motion simulation projects typically rely on rotation and translation transformations to mimic natural movement influenced by wind [4]. By applying geometric transformations to simple shapes, researchers demonstrate how realistic motion can be achieved without complex physics engines. These projects serve as effective learning tools for understanding transformation matrices and rotational motion in computer graphics.

Mechanical animation is another widely studied topic, with ferris wheel simulations being a common example. Ferris wheel animation projects use circular geometry and trigonometric functions to calculate the positions of rotating cabins [5]. Rotation transformations are applied continuously to simulate real-world amusement rides. These studies illustrate how sine and cosine functions can be used to position objects along circular paths, making them valuable references for projects involving rotational motion and repetitive animation patterns.

Environmental simulation is also an important aspect of graphical scene design. Several studies explore day and night environment transitions by dynamically changing background colors, lighting conditions, and object visibility [6]. Such projects

demonstrate how conditional rendering techniques can be used to alter the scene based on user input or time-based triggers. Day–night simulations enhance realism and allow developers to showcase multiple visual states within a single scene, which is particularly effective in cultural and environmental storytelling applications.

Weather effects, such as rain simulation, have also been addressed in prior research. Simple rain effects are often implemented using line primitives to represent falling raindrops [7]. By repeatedly rendering short line segments with downward motion, these projects create the illusion of rainfall. Timing control and conditional rendering are key aspects of such simulations, enabling rain effects to be toggled on or off based on user interaction. These techniques are especially relevant for projects that aim to depict dynamic environmental changes.

User interaction plays a crucial role in modern computer graphics applications. Many OpenGL-based projects emphasize keyboard-controlled interaction, allowing users to control object movement, animation states, or environmental changes [8]. Such works demonstrate the use of event-handling mechanisms provided by GLUT to respond to key presses in real time. Interactive features significantly improve user engagement and transform static scenes into dynamic simulations, making them an essential component of educational graphics projects.

Designing reusable and modular graphical components is another common theme in related works. Fair stall design projects often focus on constructing stalls using basic primitives while organizing code into reusable functions [9]. This modular approach improves code readability, maintainability, and scalability. By reusing drawing functions for multiple stalls, developers can efficiently expand scenes without duplicating code, which is a key software engineering principle applied within computer graphics development.

Finally, several studies focus on complete cultural event visualization by combining multiple graphical elements such as people, vehicles, decorations, and environmental effects into a single cohesive scene [10]. These works emphasize narrative-driven scene composition, where visual elements work together to represent social activities and cultural identity. Such projects demonstrate how computer graphics can be used not only for technical demonstrations but also for cultural expression and digital storytelling.

In addition to research papers and project-based studies, academic course materials play a significant role in shaping graphics

projects. The AIUB Computer Graphics course material provides structured guidance on OpenGL fundamentals, including drawing primitives, transformations, animation techniques, interaction handling, and scene rendering [11]. These materials serve as an academic foundation for student projects and directly influence the design and implementation strategies used in OpenGL-based scene simulations.

Overall, the reviewed works collectively highlight the importance of OpenGL primitives, transformation techniques, animation logic, and user interaction in building engaging 2D graphical scenes.

3. Implementation

This project is implemented using C++ and the OpenGL (GLUT) library to create a complete two-dimensional village festival scene inspired by Bengali cultural celebrations such as Pohela Boishakh. The implementation focuses on structured scene rendering, modular function design, animation using timers, and interactive control through keyboard and mouse input. The entire scene is drawn using OpenGL primitives such as points, lines, triangles, quadrilaterals, and polygons.

3.1 Overall Scene Structure

- The entire scene is rendered inside a single display function named `DrawAllComponents()`.
- This function is responsible for drawing all visual elements in a proper layered order.
- An orthographic projection is set using `gluOrtho2D(-1000, 1000, -1000, 1000)` which allows precise object placement using Cartesian coordinates.
- The scene is logically divided into the following regions:
 - Sky
 - Hills
 - Ground
 - Water (river)
 - Roads
 - Houses
 - Fair stalls
 - Crowd and people
 - Vehicles
 - Animated objects
- `glPushMatrix()` and `glPopMatrix()` are used extensively to:
 - Isolate transformations
 - Prevent unwanted effects on other objects
- Translation, scaling, and rotation are applied locally to individual objects.

3.2 Key Visual Features

3.2.1 Sky, Sun, Moon, and Stars

- The sky color changes based on environmental conditions.
- During daytime:
 - A light blue sky is rendered.
- During night:
 - The sky becomes dark blue.
 - Stars appear in the sky.
- During rain:
 - The sky color becomes darker to simulate cloudy weather.
- Stars are drawn only in night mode using:
 - A fixed array of star positions `(STAR_POS)`
 - `GL_POINTS` primitive
- The sun is drawn during daytime using a solid circular function.
- The moon is drawn during night using:
 - Two overlapping circles to create a crescent shape.

3.2.2 Clouds and Weather

- Clouds are created using multiple overlapping circles.
- A single cloud is drawn using the `drawSingleCloud()` function.
- Multiple clouds are grouped using the `drawCloudGroup()` function.
- Translation and scaling are applied to vary cloud position and size.
- Cloud color changes automatically based on:
 - Day or night
 - Rain condition
- Rain is implemented using the `drawRainSimple()` function.
- Rain effect consists of:
 - Multiple diagonal line segments
 - Continuous downward motion
- Rain is conditionally rendered only when rain mode is enabled.

3.2.3 Hills, Ground, and Water

- Hills are drawn using triangles and polygons.
- Different shades of green are used to create depth perception.
- The ground is drawn as a large polygon covering the middle area.
- The river occupies the lower part of the scene.
- The river includes animated boats that move continuously.

3.2.4 Roads and Dash Line System

- The scene includes:
 - One main vertical road
 - One diagonal side road
- Roads are drawn using quadrilateral shapes with dark gray color.
- Yellow dashed center lines are drawn on the roads.
- For the main road:
 - Dash lines are created using a loop along the y-axis.
 - A perspective narrowing effect is applied.
- For the side road:
 - Linear interpolation is used between midpoints of the road edges.

- Dash lines are carefully aligned to stay inside the road.
- Dash lines do not cross into water or land areas.

3.2.5 Human Figures and Crowd

- Human figures are drawn using:
 - Circles
 - Quads
 - Triangles
 - Lines
- Separate functions are used to draw:
 - Boys
 - Girls
 - Men
 - Women
 - Balloon sellers
 - Playing children
- Clothing styles represent traditional Bengali attire:
 - Saree
 - Kurta
- Crowd placement uses:
 - Loops
 - Fixed spacing logic
- Scaling is applied to:
 - Create variation in size
 - Simulate depth

3.2.6 Fair Stalls and Shops

- Fair stalls are implemented using modular functions.
- Each stall contains:
 - Base
 - Counter
 - Roof
 - Hanging flags
 - Decorative elements
- Multiple shops are drawn using translation.
- This demonstrates:
 - Code reuse
 - Structured design
- Balloon shops and side stalls add visual richness.
- Balloons are drawn using:
 - Polygons
 - Line-based strings

3.3 Animation and Moving Objects

Animation is a core feature of this project and is implemented using timer-based updates and geometric transformations. Several objects in the scene are animated to create a lively and realistic village festival environment.

3.3.1 Cloud Animation

Cloud animation plays a major role in enhancing the realism of the sky. Instead of using static clouds, the project implements continuously moving clouds to simulate natural atmospheric motion.

- Clouds are constructed using multiple overlapping solid circles.
- A single cloud is drawn using the function `drawSingleCloud()`
- Multiple clouds are grouped together using the function `drawCloudGroup(x, y, scale)`.
- Translation and scaling are applied so that cloud groups appear at different heights and sizes.

Cloud movement is controlled by a global variable named `moveC`.

- The timer function `updateC()` is responsible for updating the cloud position.
- In each timer cycle, the value of `moveC` is increased gradually.
- This creates smooth left-to-right horizontal motion across the sky.
- When clouds move outside the visible screen boundary, their position is reset.
- This reset creates a continuous looping animation effect.

Cloud appearance also reacts to environmental conditions:

- During daytime, clouds are rendered in white color.
- At night, cloud color changes to light gray.
- When rain mode is enabled, clouds become darker to represent heavy rain clouds.

This interaction between cloud animation, day–night mode, and rain mode adds realism and improves the visual quality of the scene.

3.3.2 Boat Animation

Two boats are animated on the river using translation transformations.

- Boat movement is controlled using timer functions `updateB1()` and `updateB2()`.
- One boat moves horizontally with adjustable speed.
- Another boat moves continuously in the opposite direction.
- Keyboard and mouse input allow speed and position control of boats.

3.3.3 Ferris Wheel Animation

Two ferris wheels are animated using rotational motion.

- Each wheel rotates around its center using a rotation angle.
- The rotation angle is updated using the `updateW()` timer function.
- Trigonometric functions (sine and cosine) are used to position the gondolas correctly.
- Rotation can be paused and resumed using keyboard input.

3.3.4 Bird Animation

Birds are animated to fly across the sky.

- Horizontal motion is achieved by updating the x-position.
- Vertical bobbing is added using a sine wave.
- This creates a natural flying motion.

3.3.5 Car Animation

A car moves along the road using translation and scaling.

- As the car moves upward, its scale gradually decreases.

- This creates a depth perspective effect.
- When the car reaches the end of the road, it resets to its starting position.
- The update function use for this `updateD()`.

3.4 Interactive Controls (Keyboard and Mouse)

The project supports the following interactions:

- Press `n` to enable night mode (moon and stars appear)
- Press `d` to switch back to day mode
- Press `x` to stop ferris wheel rotation
- Press `s` to resume ferris wheel rotation
- Press `spacebar` to toggle rain:
 - Dark sky
 - Rain animation
 - Umbrella appears on people
- Arrow keys:
 - -Move the kite
 - -Scale the kite
- Mouse left click:
 - Decreases boat speed
- Mouse right click:
 - -Increases boat speed
- Keys `L` and `R`:
 - Adjust boat position

These controls make the scene dynamic and user-controlled.

3.5 Key Technologies Used

- C++ programming language
- OpenGL fixed-function pipeline
- GLUT library for:
 - Window creation
 - Input handling
 - Timer functions
- Trigonometric functions: `sin` and `cos`
- Timer-based animation using `glutTimerFunc`

3.6 Summery table of Functions Used

No.	Function Name	Purpose / What it Does
1	drawSolidCircle2D(cx, cy, r)	Draws a filled 2D circle using triangle fan technique
2	drawCircle2D(cx, cy, r)	Draws a smooth circle used for sun, moon, and sky objects
3	drawCircle(cx, cy, r, seg)	Draws a general circle with adjustable smoothness
4	drawSingleCloud()	Draws a single cloud with color changes based on weather
5	drawCloudGroup(x, y, scale)	Draws a group of clouds using translation and scaling
6	drawTreeCloudTop(x, y, scale)	Draws a tree using cloud-like leaves and a trunk
7	ferrisWheel1(moveX, moveY)	Draws and animates the first ferris wheel
8	ferrisWheel2(moveX, moveY)	Draws and animates the second ferris wheel
9	drawFairStall(cx, cy, scale)	Draws a complete fair stall with roof, counter, and flags
10	drawLatim(x, y, scale)	Draws and rotates a traditional spinning toy (latim)
11	drawCar()	Draws a moving car with scaling for depth effect
12	drawComingCar()	Draws a car approaching from distance using perspective
13	drawBoy(x, y)	Draws a boy character using basic shapes
14	drawMale()	Draws a simple male human figure
15	drawGirl(x, y)	Draws a girl character with hair and dress
16	drawWoman(x, y)	Draws a woman wearing traditional clothing
17	drawBoyGirlPair(x, y)	Draws a boy and a girl together as a pair
18	drawHuman(x, y)	Draws a generic human figure
19	drawPBoy(x, y, scale)	Draws a playing boy character
20	drawPersons()	Draws multiple people using translation
21	drawBengaliCrowd(startX, scale)	Draws a crowd of people using loops and spacing
22	drawBalloonSeller()	Draws a balloon seller with balloons and strings
23	drawRainSimple()	Draws rain animation using line primitives
24	drawUmbrella(x, y)	Draws an umbrella when rain mode is active
25	drawBirdV(x, y, s)	Draws flying birds with simple animation
26	drawSerialShops()	Draws multiple shops along the X-axis
27	DrawAllComponents()	Main function that renders the entire scene
28	updateC(int)	Animates cloud movement across the sky
29	updateB1(int)	Controls movement and speed of the first boat
30	updateB2(int)	Animates the second boat in the river
31	updateW(int)	Rotates the ferris wheels continuously
32	updateMoon(int)	Animates moon rising during night mode
33	updateD(int)	Animates car movement and scaling
34	updateBirds(int)	Animates bird movement and flying motion
35	handleKeypress(key, x, y)	Handles keyboard input for modes and animations
36	handleSpecialKey(key, x, y)	Handles arrow key input for kite movement
37	handleMouse(button, x, y)	Controls boat speed using mouse clicks
38	display()	Calls the main drawing function and refreshes screen
39	init()	Initializes OpenGL settings
40	main(argc, argv)	Program entry point that starts the OpenGL loop

Table 2: List of Key Functions Used in the Project

4. Screenshots of the Project

Scenario 1: Day scene overview

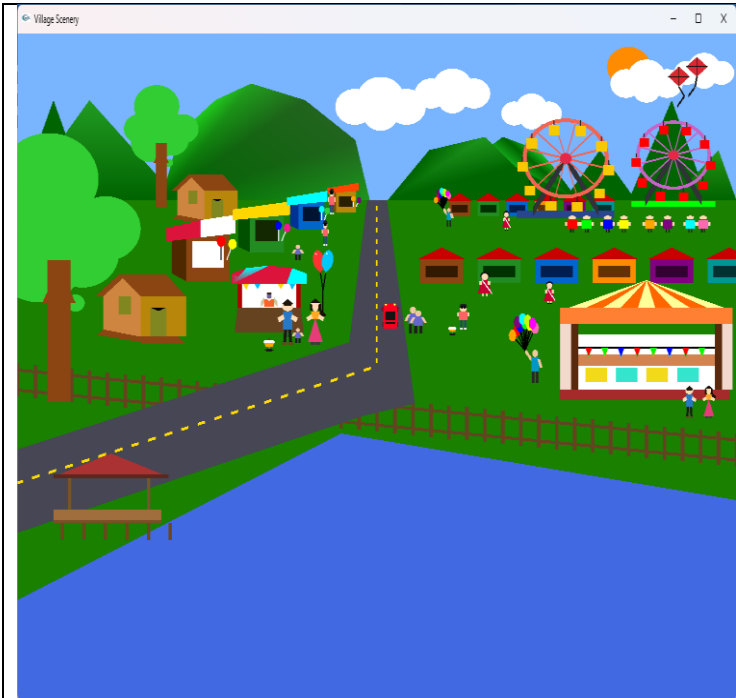


Figure: 01

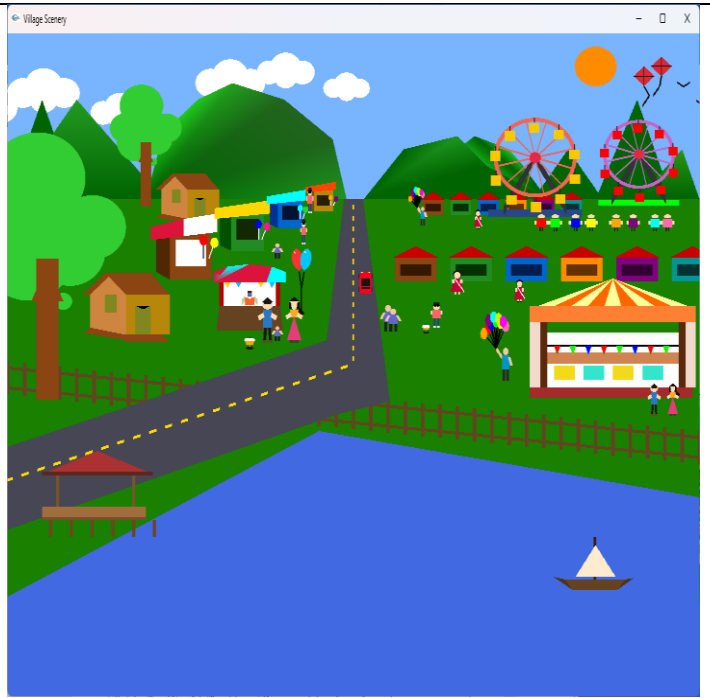


Figure: 02

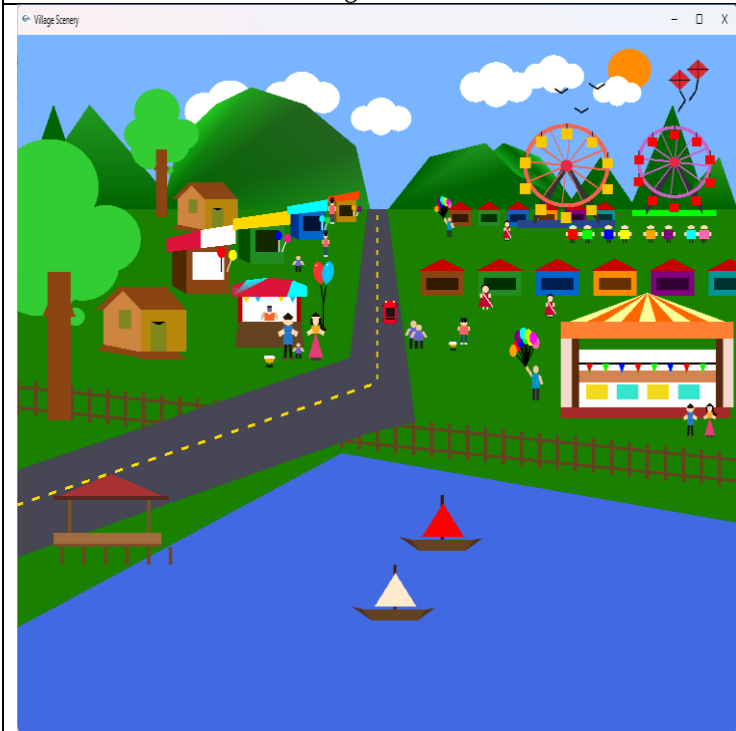


Figure: 03

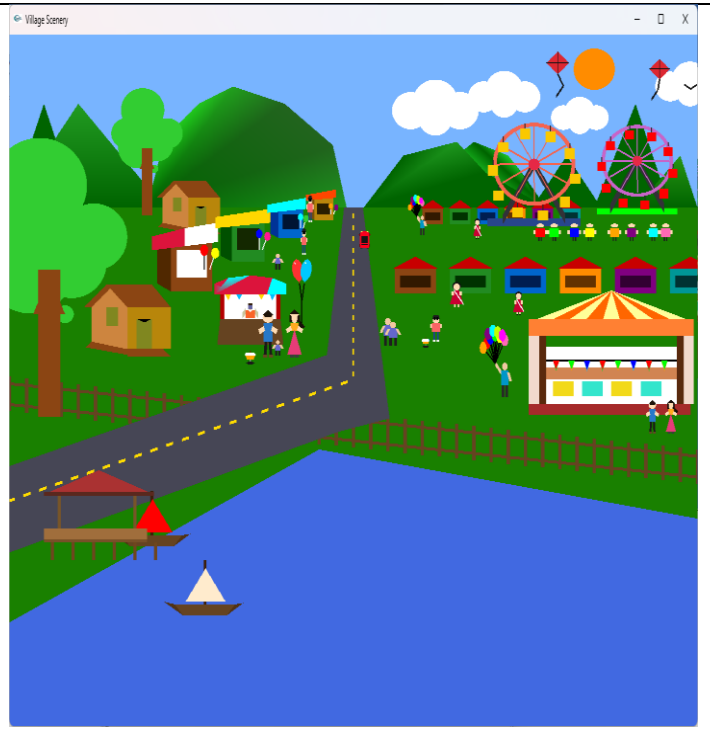


Figure: 04

Table 03: Day Scene Overview

Scenario 2: Night scene with stars and moon

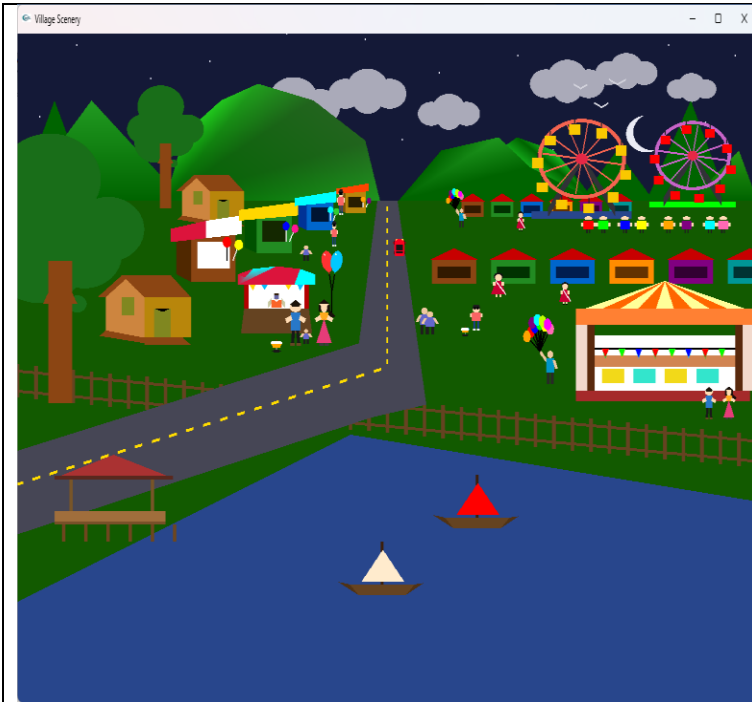


Figure: 01

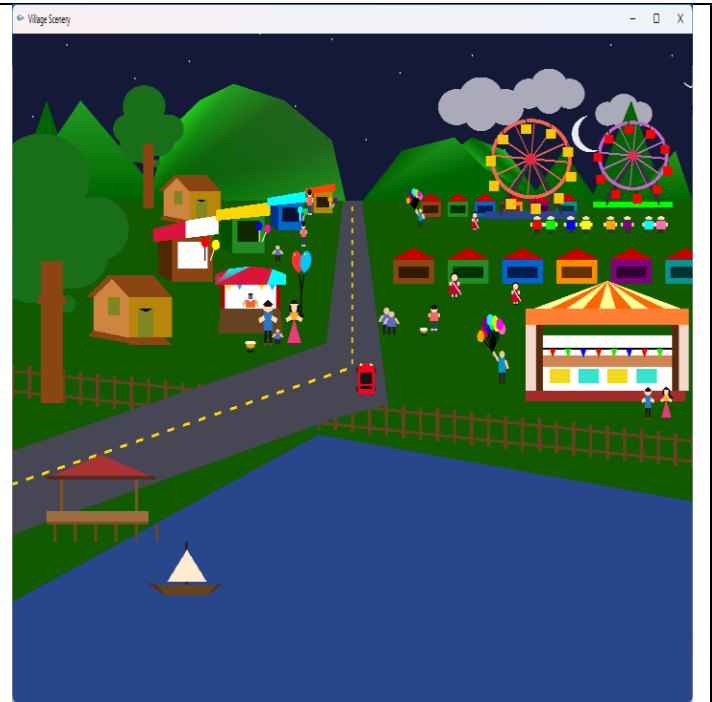


Figure: 02

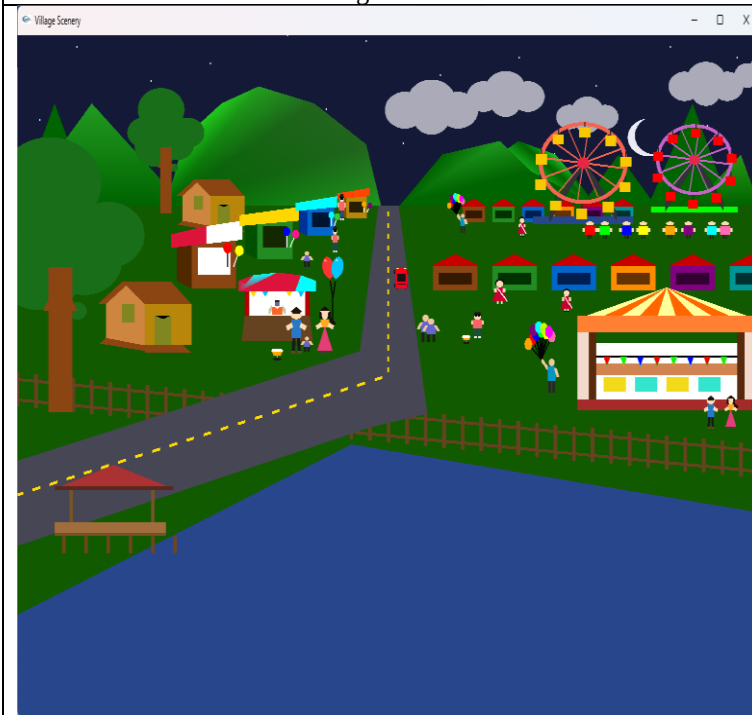


Figure: 03

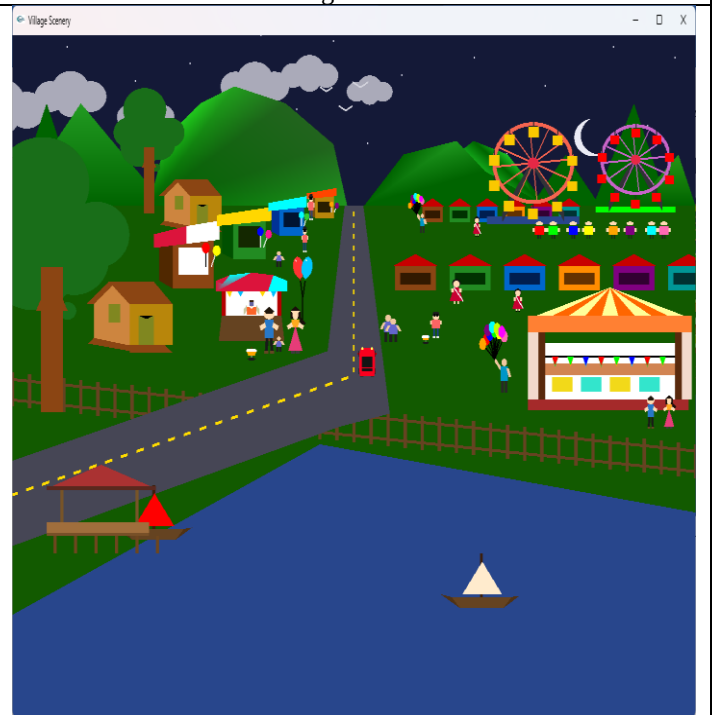


Figure: 04

Table 04: Night Scene Overview

Scenario 3: Day Scene Overview when Rain occur

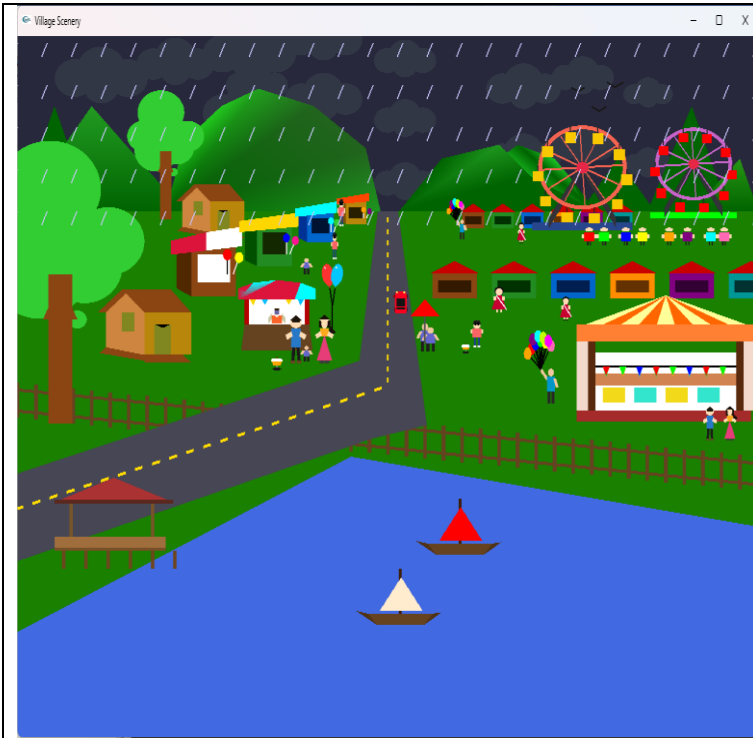


Figure: 01

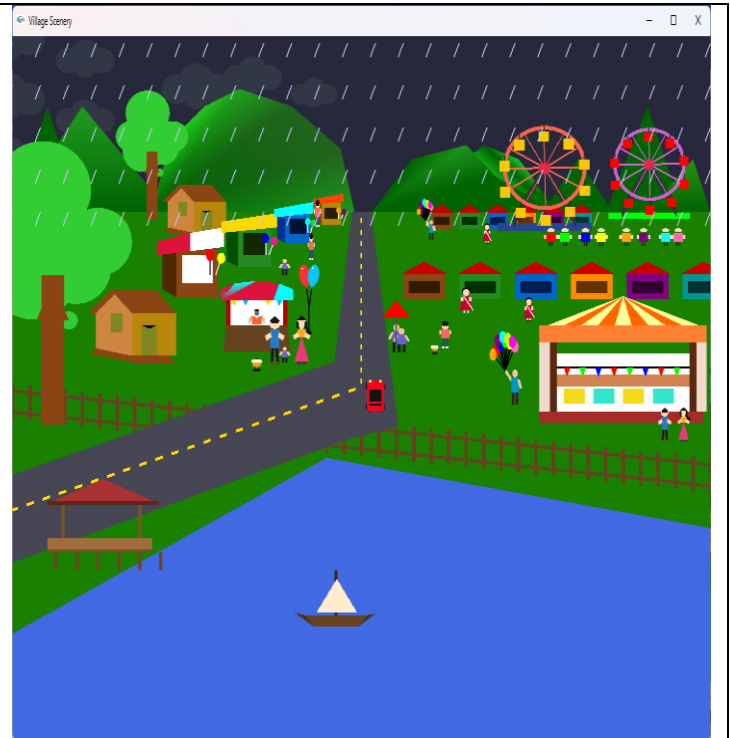


Figure: 02

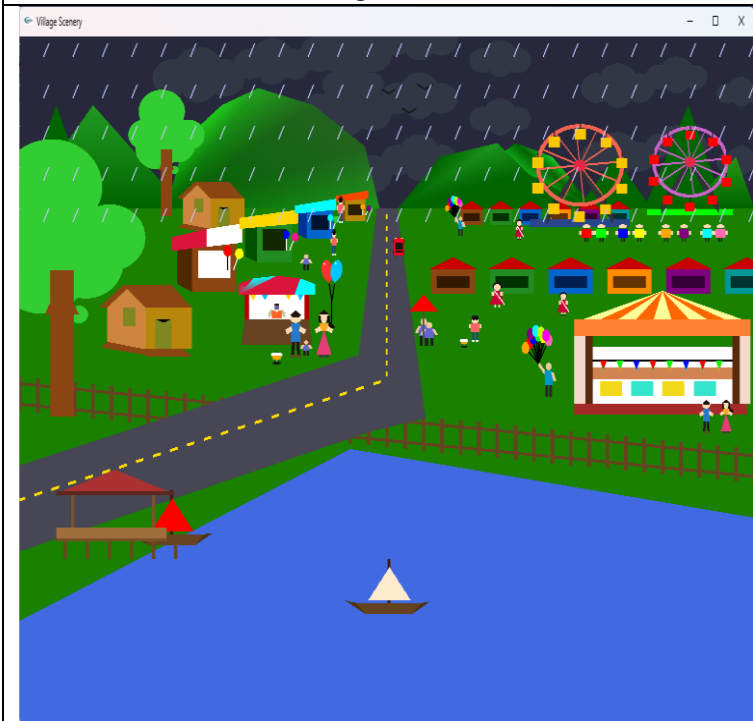


Figure: 03

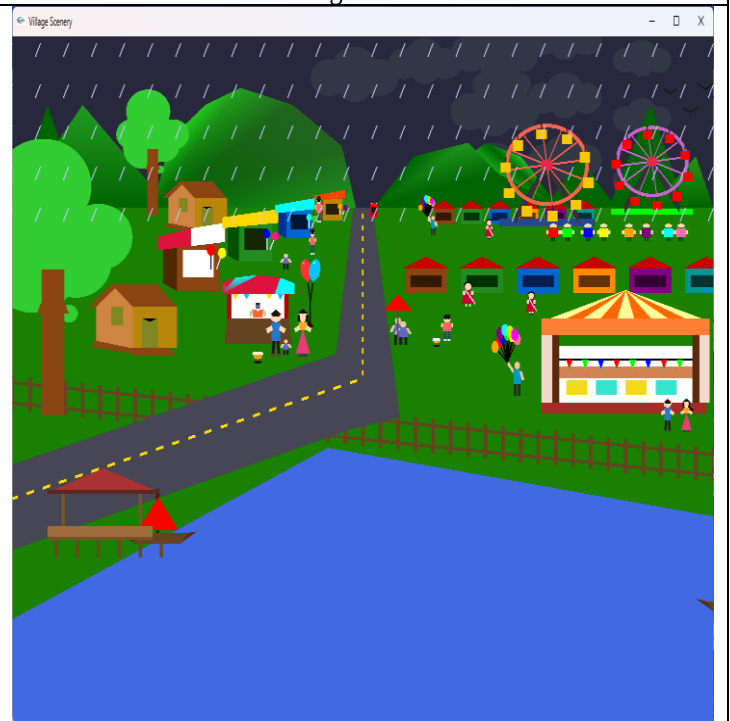


Figure: 04

Table 05: Rain Scene (Day)

Scenario 4: Night Scene Overview when Rain Occur

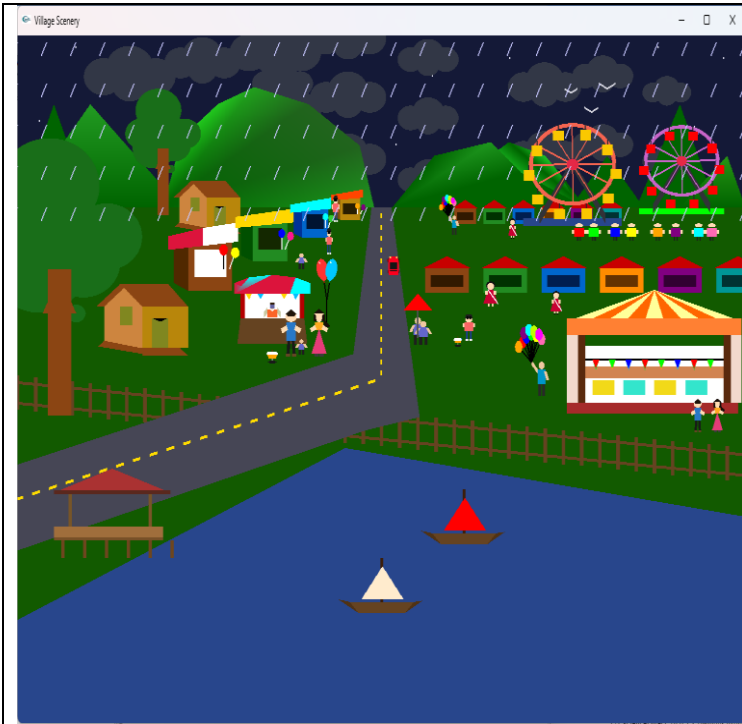


Figure: 01

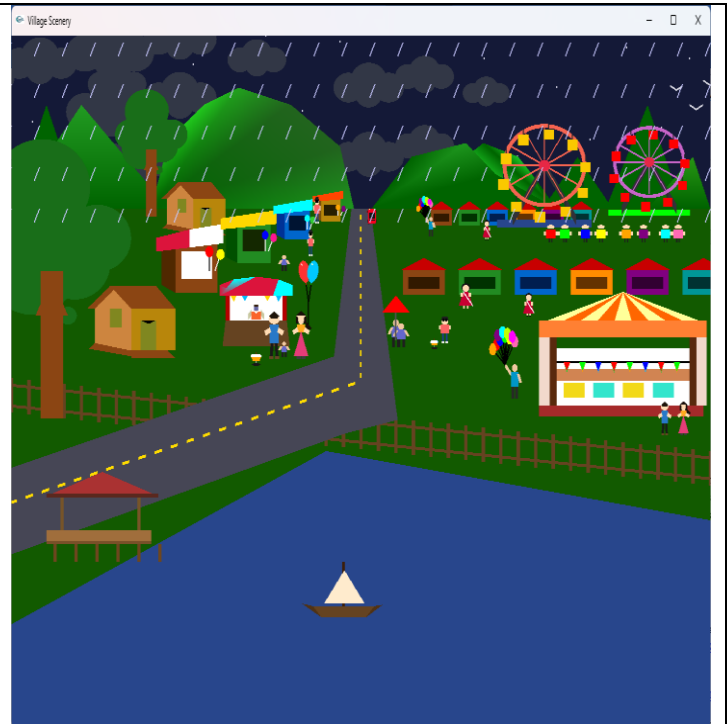


Figure: 02

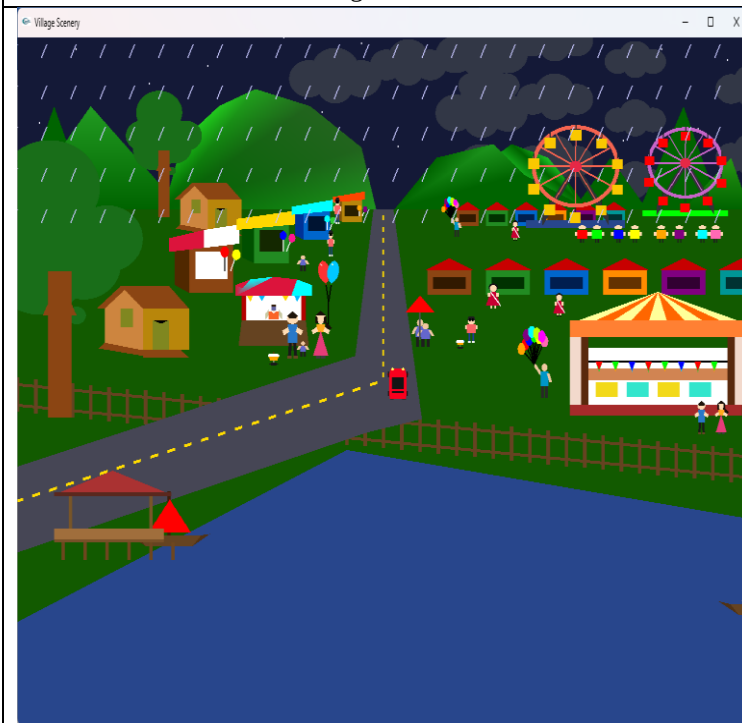


Figure: 03

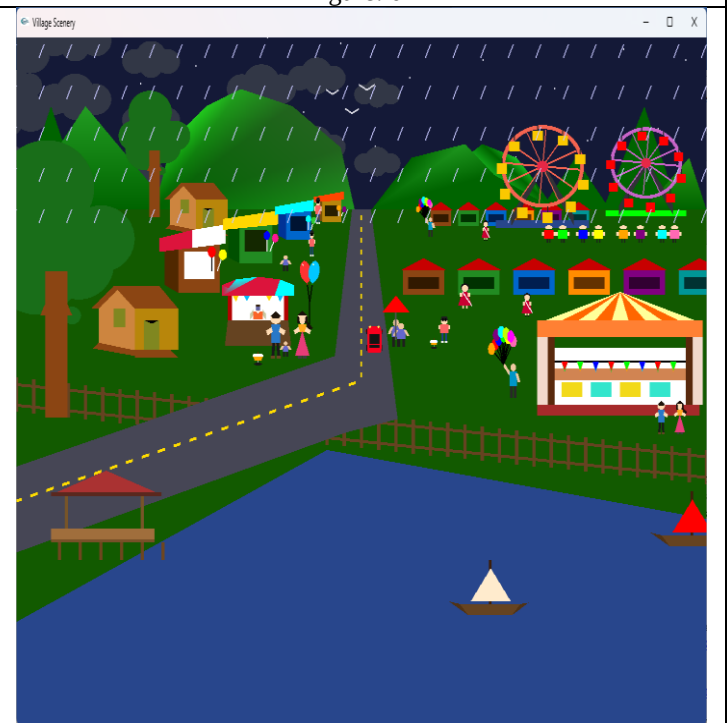


Figure: 04

Table 06: Rain Scene (Night)

5. Code Availability

GitHub Repository Link:

<https://github.com/fihabefty114/Animated-Pohela-Boishakh-Festival-Scene.git>

6. Future Work

In future versions of this project, several improvements can be made to enhance realism and user experience. Sound effects and background music can be added to create a more immersive festival atmosphere, such as crowd noise, river sounds, and traditional Bengali music. Character animation can be improved by introducing walking movements, hand gestures, and more natural motion. Textures and lighting effects can be applied to objects like houses, roads, and stalls to increase visual realism. The entire scene can also be converted into a three-dimensional environment to provide depth perception and camera movement. Additionally, performance optimization techniques such as reducing draw calls, improving animation efficiency, and managing object complexity can be applied to ensure smoother rendering on different systems.

7. Conclusion

This project demonstrates how OpenGL can be used to create a complete animated cultural scene. Although the project has limitations such as lack of textures and advanced lighting, it successfully achieves its goal of combining animation, interaction, and cultural representation in a single project.

Although the project has limitations such as the absence of textures, advanced lighting, and realistic physics, it successfully achieves its main objective of integrating animation, interactivity, and cultural representation within a single OpenGL-based application. The project also serves as a strong learning example for understanding scene composition, coordinate systems, timer-based animation, and input handling in computer graphics, and it provides a solid foundation for future enhancements and more advanced graphical implementations.

8. References

- [1] LearnOpenGL. (n.d.). *Coordinate systems*. Retrieved from <https://learnopengl.com/Getting-started/Coordinate-Systems>
- [2] ScienceDirect. (n.d.). *Computer graphics*. <https://www.sciencedirect.com/topics/computer-science/computer-graphics>
- [3] OpenGL Architecture Review Board. (n.d.). *GLUT specification*. <https://www.opengl.org/resources/libraries/glut/spec3/node1.html>
- [4] Weisstein, E. W. (n.d.). *Rotation transformation*. MathWorld—A Wolfram Web Resource. <https://mathworld.wolfram.com/RotationTransformation.html>

- [5] LearnOpenGL. (n.d.). *Transformations*. Retrieved from <https://learnopengl.com/Transformations>
- [6] NVIDIA Corporation. (2004). *GPU Gems: Programming techniques, tips, and tricks for real-time graphics*. <https://developer.nvidia.com/gpugems/gpugems>
- [7] Bell, J. (n.d.). *Computer graphics course notes*. University of Illinois at Chicago. <https://www.cs.uic.edu/~jbell/CourseNotes/ComputerGraphics/>
- [8] OpenGL Architecture Review Board. (n.d.). *GLUT keyboard and special key handling*. <https://www.opengl.org/resources/libraries/glut/spec3/node49.html>
- [9] Emmanuel, A. (2013). *Introduction to computer graphics*. Worcester Polytechnic Institute. <https://web.cs.wpi.edu/~emmanuel/courses/cs543/f13/slides/lecture01.pdf>
- [10] Tutorialspoint. (n.d.). *Computer graphics tutorial*. https://www.tutorialspoint.com/computer_graphics/index.htm
- [11] American International University–Bangladesh (AIUB). (n.d.). *Computer graphics course materials (CSE)*. [Computer Graphics Repository - OneDrive](#)